BCP 210: Data Structures and Algorithms I

Coursework Assignment 2 — Written Report

Part A: Algorithmic Complexity Analysis

A1. Worst-case complexity of algorithm_x

algorithm_x uses two nested loops: the outer loop runs i from 0 to N-1, and for each i the inner loop runs j from i to N-1. In the worst case (no matching pair, or the match found only at the very last comparison), the total number of (i, j) pairs examined is proportional to N(N-1)/2. The nested loop structure is what drives the cost, giving a worst-case time complexity of **O(N^2)**.

A2. Worst-case complexity of algorithm_y

algorithm_y makes a single pass through the records (one loop, O(N) iterations), and for each value it does an O(1) average-case dictionary lookup/insert using the hash table `seen`. This gives an overall worst-case time complexity of **O(N)**. The hash table (Python dict) is what makes it faster than algorithm_x: it allows checking 'have I seen the complement before?' in O(1) time instead of re-scanning the array. The trade-off is space: algorithm_y uses **O(N)** additional space to store the `seen` dictionary, whereas algorithm_x uses only O(1) extra space.

A3. Algorithm Z

algorithm_z is **Insertion Sort**. Its best-case time complexity is **O(N)**, which occurs when the input array is already sorted (or nearly sorted) — the inner while loop condition `records[j] > key` fails immediately for each i, so only one comparison per element is needed. Its worst-case time complexity is **O(N^2)**, which occurs when the input array is sorted in reverse (descending) order — every new element must be compared against and shifted past all previously sorted elements.

A4. Operations at N = 1,000,000

Complexity Class	Approx. Operations at N = 10^6	Rank (1 = fastest)
O(1)	1	1
O(log N)	~20	2
O(N)	1,000,000	3
O(N log N)	~20,000,000	4
O(N^2)	1,000,000,000,000	5

A5. Fibonacci recursion

The naive recursive f(n) = f(n-1) + f(n-2) is O(2^N) because each call that is not a base case spawns TWO further recursive calls, and this branching repeats down to depth N. The resulting call tree has exponentially

many nodes (each level roughly doubling the number of calls), and crucially the SAME sub-problems (e.g. f(n-3)) are recomputed many times over because no results are cached. To reduce this to O(N) time and O(1) space, replace the recursion with an iterative loop that keeps only the last two values: start with a=0, b=1, and for each step from 2 to n set (a, b) = (b, a+b), returning b at the end. This computes each Fibonacci number exactly once, using a fixed number of variables regardless of n.

Part B: Arrays and the Two-Pointer Technique

Full implementations are provided in **src/part_b_arrays.py**: **binary_search**, **find_pair_with_sum**, and **rotate_array**.

B1. binary_search complexity

Best case: **O(1)** — the target happens to be at the middle index on the very first check. Worst case: **O(log N)** — the target is at one extreme of the array or absent altogether, forcing the search interval to be halved repeatedly until only one element remains.

B2. find_pair_with_sum demonstration

Using catalogue = [-8, -3, 0, 1, 4, 6, 9, 12, 15, 21] and target = 13, the two pointers start at the ends and move inward based on whether the current sum is too small or too large. The function returns the pair (-8, 21), since -8 + 21 = 13 and the left/right pointers meet this sum before any other combination. (Program output confirms this — see src/part_b_arrays.py.)

B3. rotate_array demonstration

rotate_array([1, 2, 3, 4, 5], 2) correctly produces [4, 5, 1, 2, 3] using the triple-reversal approach: reverse the whole array, then reverse the first k elements, then reverse the remaining N-k elements.

B4. Amortised O(1) append

A Python list is backed by a fixed-size dynamic array that is over-allocated with some spare capacity. Most append() calls simply write into the next free slot, which is O(1). Occasionally, when the underlying array is full, Python must allocate a new, larger block of memory and copy all N existing elements into it — this single append costs O(N). However, because the array's capacity grows geometrically (roughly by a constant factor each time it resizes), these expensive O(N) resizes become progressively rarer as the list grows. Averaged ('amortised') over a long sequence of N appends, the total cost is O(N), so each individual append costs O(1) on average — even though any single append might occasionally be O(N).

Part C: Linked Lists, Stacks, and Queues

Full implementations are provided in **src/part_c_shuttle.py**: the **Booking** node class, **ShuttleList** (add_booking, cancel_booking, find_and_swap), **RouteHistory**, and **BoardingQueue**.

C2. add_booking complexity

With a maintained tail pointer, add_booking runs in **O(1)**, since the new node can be linked directly after self.tail without traversal. Without a tail pointer, the list would have to be walked from the head to find the last node, making the operation **O(N)**.

C4. find_and_swap complexity and design choice

find_and_swap runs in **O(N)**: locating the two target nodes requires scanning the list (at most N comparisons), while the swap itself is O(1). Swapping the data fields is preferred over relinking next/prev pointers because pointer relinking in a doubly linked list requires updating up to eight separate pointers correctly (the two nodes' own next/prev plus their neighbours' next/prev), with extra edge cases when the nodes are adjacent or are the head/tail of the list. Swapping three data attributes is simpler and far less error-prone.

C5. RouteHistory operation complexities

push: **O(1)** amortised (list.append). pop_undo: **O(1)** (list.pop from the end). peek: **O(1)** (direct index into the last element).

C6. Why deque over a plain list for a queue

A plain Python list is a dynamic array, so removing from the FRONT (list.pop(0)) is **O(N)** because every remaining element must shift left by one slot. collections.deque is implemented internally as a doubly linked sequence of fixed-size blocks, giving **O(1)** performance for appends and pops at BOTH ends. Since a boarding queue constantly adds at the back and removes from the front, deque is the structurally correct and efficient choice.

Part D: Hash Tables and Binary Search Trees

Full implementations are provided in **src/part_d_grades.py**: **grade_frequency_report**, **find_students_with_grade**, **insert**, **inorder_traversal**, **search**, and **find_range**.

D1. Hash collisions

A hash collision occurs when two different keys hash to the same slot (index) in the underlying array of a hash table. **Chaining** resolves this by storing a small list (or linked list) of all key-value pairs that map to that slot; worst-case lookup is **O(N)** if all N keys collide into a single slot. **Open addressing / linear probing** resolves collisions by searching forward through the array for the next free slot when a collision occurs; worst-case lookup is also **O(N)**, since in a pathological case a lookup may have to probe through nearly every occupied slot in the table before finding the key or an empty slot.

D3. Why find_students_with_grade is O(N log N), not O(N)

Grouping students by grade using a hash-based pass (a single scan collecting matching student_ids into a list) is O(N). However, the function is required to return the ids in **sorted** order, and sorting a list of K matches with Python's built-in sort (Timsort) costs O(K log K). In the worst case K is proportional to N, so the overall complexity is **O(N log N)** — the hash table speeds up grouping, but it cannot avoid the

comparison-sort needed to order the final result.

D4. BST built from records

Inserting (1001, 72), (1002, 55), (1003, 88), (1004, 60), (1005, 95), (1006, 48) in order, keyed on grade_score, produces the following tree (shown as an indented structure):

```
72 (1001)
    ■■■ left: 55 (1002)
        ■■■ left: 48 (1006)
        ■■■ right: 60 (1004)
    ■■■ right: 88 (1003)
        ■■■ right: 95 (1005)
```

D5. In-order traversal output

Calling inorder_traversal on the tree above yields, in ascending order of grade_score: (1006, 48), (1002, 55), (1004, 60), (1001, 72), (1003, 88), (1005, 95). In-order traversal (left subtree, node, right subtree) produces a sorted sequence because of the BST invariant: every node in a left subtree has a strictly smaller key than its parent, and every node in a right subtree has a key greater than or equal to its parent. Recursively visiting left-before-self-before-right at every level therefore guarantees non-decreasing key order across the whole traversal.

D6. search and find_range complexity

Both search and find_range run in terms of the tree height H: **search** is **O(H)**, since at each node it discards one entire subtree and descends one level. **find_range** is **O(H + K)**, where K is the number of nodes that actually fall within [low, high] — H accounts for descending to the range boundaries, and K accounts for visiting and collecting the matching nodes; subtrees entirely outside the range are pruned and never visited. H = O(log N) when the tree is reasonably balanced (each subtree splits the remaining nodes roughly evenly). H = O(N) in the worst case, when insertions arrive in already-sorted (or reverse-sorted) order, causing the tree to degenerate into a single chain with no branching, equivalent to a linked list.

Part E (Bonus): Exam Invigilation Scheduler

Full implementations are provided in **src/part_e_bonus.py**: **build_conflict_map** and **detect_first_conflict**.

E1. Sample output

Running build_conflict_map on the sample sessions produces: {'R1': [(0, 1)], 'R2': [(3, 4)]} — matching the expected conflicts described in the assignment (session 0 vs 1 in R1, session 3 vs 4 in R2).

E3. Complexity discussion

The naive approach groups sessions by room in O(N) (one pass with a hash table), then for a room with M sessions compares all pairs in O(M^2). Summed across all rooms, the worst case (all N sessions in one room) is **O(N^2)** time and O(N) space for the grouping structure plus O(C) for the C conflicts found. This can

be improved: if sessions within each room are first sorted by start time (O(M log M) per room, O(N log N) overall) and then swept with a stack as in detect_first_conflict, each session is pushed and popped at most once, giving O(M) per room after sorting — O(N log N) overall, a significant improvement over O(N^2) for rooms with many sessions. A BST keyed on start time would offer similar O(log M) insertion/lookup per session but doesn't by itself detect interval overlaps without extra augmentation (e.g. storing max end-time in each subtree, as in an interval tree); a sorted array with binary search is simpler to implement here and, combined with the sweep, achieves the same O(N log N) overall bound with less complexity than a full interval tree.